



NUEVAS PROPIEDADES CSS3

UD 02 - HOJAS DE ESTILOS (III)



ÍNDICE

- SOMBRAS DE TEXTO
- TRANSFORMACIONES
- FONDOS MÚLTIPLES
- COLUMNAS MÚLTIPLES
- OPACIDAD
- TRANSPARENCIAS. RGBA()



SOMBRAS DE TEXTO

La propiedad text-shadow nos permite definir una sombra a un texto, la sintaxis más común es:

```
Elemento {  
  text-shadow: desplazamientoX desplazamientoY radio-de-  
desenfoque color;  
}
```

La propiedad tiene 4 valores que son los siguientes:

1. El desplazamiento horizontal(eje X) de la sombra, un desplazamiento negativo pondrá la sombra a la izquierda.
2. El desplazamiento vertical (eje Y), un valor negativo dispone la sombra en la parte superior del texto, uno positivo la sombra estará por debajo del texto.
3. El tercer parámetro es el radio de desenfoque, si se pone a 0 la sombra será fuerte y con color liso, cuanto más grande el número, más borrosa será.
4. El último parámetro es el color de la sombra.



SOMBRAS DE TEXTO

Ejemplo:

Queremos que un texto tenga una sombra en la parte inferior a derecha con un pequeño desenfoque de color gris luego debemos implementar el siguiente código:

```
#titulo1 {  
  text-shadow: 5px 5px 5px #aaa;  
}
```

Titulo sombreado



SOMBRAS DE TEXTO

Ejemplo:

Queremos que la sombra se disponga en la parte superior izquierda de cada letra luego debemos definir los siguientes valores:

```
#titulo2 {  
  text-shadow: -5px -5px 5px #aaa;  
}
```

Titulo sombreado



SOMBRAS DE TEXTO

Otra sintaxis de text-shadow es aplicar varias sombras al texto, por ejemplo:

```
#titulo3 {  
  text-shadow: 3px 3px 5px #f00,  
              6px 6px 5px #0f0,  
              9px 9px 5px #00f;  
}
```

Titulo sombreado



TRANSFORMACIONES

<https://lenguajecss.com/css/animaciones/transformaciones/>

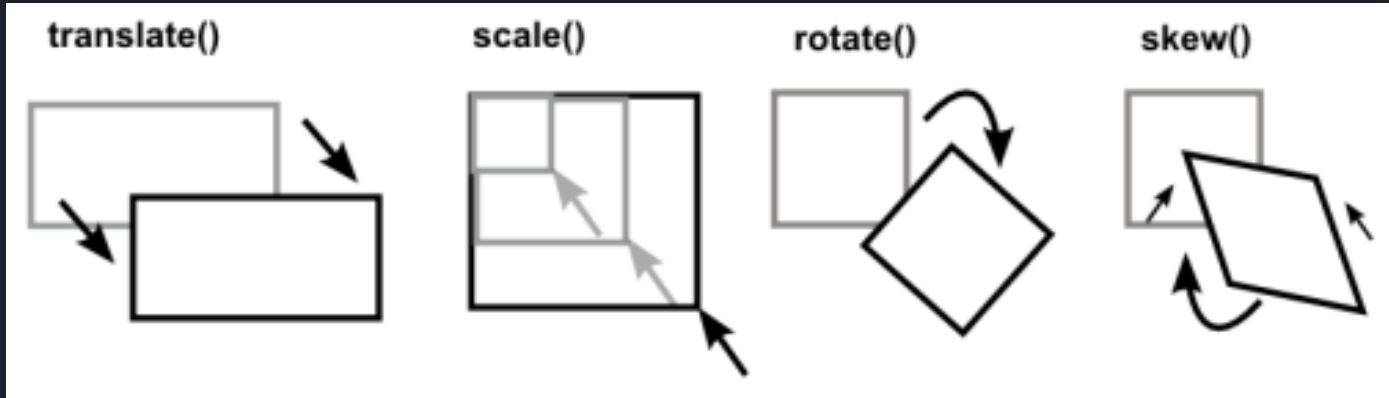
Las transformaciones es uno de los elementos más interesantes que se introducen en CSS3 para convertir el lenguaje de hojas de estilo en un sistema capaz de realizar todo tipo de efectos visuales, incluido 2D y 3D.

Tengamos en cuenta que la propiedad de transformación 2D definitiva será:

```
Elemento {  
    transform: función de transformación(valor(es));  
}
```

TRANSFORMACIONES

Estas son las transformaciones que podremos realizar:





TRANSFORMACIONES: TRANSLATE

`translate(<translation-value>[, <translation-value> ])`

Aplica una traslación 2D especificada por el vector [tx, ty]. Si no se indica el valor ty, su valor por defecto es 0.

- `TranslateX()`: aplica una traslación 2D en el eje x.
- `TranslateY()`: aplica una traslación 2D en el eje y.



TRANSFORMACIONES: SCALE

Esta función permite agrandar o reducir el tamaño del elemento. La primera sintaxis que podemos utilizar es la siguiente:

```
Elemento {  
  transform: scale(valorx ,valory);  
}
```

El primer parámetro indica la escala para **x** y el segundo parámetro es para la escala en **y**.

Con el valor 1 el elemento queda como está, con un valor mayor crece y con un valor menor decrece.



TRANSFORMACIONES: SCALE

Por ejemplo si queremos que sea del doble de ancho y la mitad de altura luego será:

```
Elemento {  
  transform: scale(2 ,0.5);  
}
```

Si tenemos que escalar solo en **x** o en **y** podemos utilizar alguna de las dos funciones **scaleX** o **scaleY**.

Otra posibilidad es utilizar un **valor negativo**, lo que nos permite tener una **reflexión** del elemento.



TRANSFORMACIONES: ROTATE

La función de rotación se llama `rotate` y tiene un `parámetro` que indica la cantidad de `grados` a rotar.

La rotación es en el `sentido de las agujas del reloj`.

Podemos indicar un valor `negativo` para rotar en sentido `antihorario`.



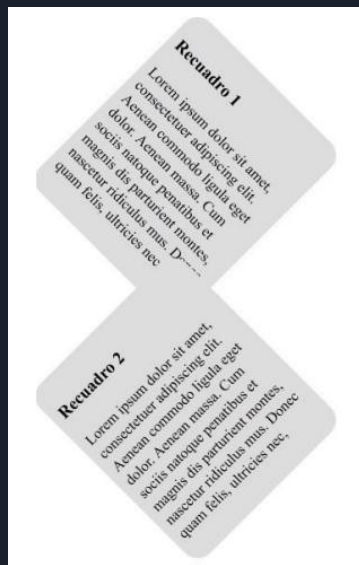
TRANSFORMACIONES: ROTATE

Para rotar un recuadro 45 grados en sentidos de las agujas de un reloj y que funcione en la mayoría de los navegadores deberemos implementar el siguiente código:

```
#recuadro1{  
  -ms-transform: rotate(45deg);  
  -webkit-transform: rotate(45deg);  
  -moz-transform: rotate(45deg);  
  -o-transform: rotate(45deg);  
  transform: rotate(45deg);  
  
  border-radius: 20px;  
  background-color: #ddd;  
  width: 200px;  
  padding: 10px;  
}
```

TRANSFORMACIONES: ROTATE

El punto de rotación coincide con el centro del recuadro (es como clavar una alfiler en el centro del recuadro y luego rotar el recuadro en el sentido de las agujas del reloj).





TRANSFORMACIONES: ROTATE

PROPIEDAD transform-origin

Podemos variar dicho punto y ubicar por ejemplo en cualquiera de los cuatro vértices del recuadro con la siguiente sintaxis:

```
Elemento {  
  transform-origin: left top;  
}
```

```
Elemento {  
  transform-origin: right top;  
}
```

```
Elemento {  
  transform-origin: left bottom;  
}
```

```
Elemento {  
  transform-origin: right bottom;  
}
```



TRANSFORMACIONES: ROTATE

Posibles valores para el x-axis (el eje horizontal):

- left (izquierda)
- center (centro)
- right (derecha)
- distancia
- %

Posibles valores para y-axis (el eje vertical) :

- top (arriba)
- center (centro)
- bottom (abajo)
- distancia
- %

El primer valor representa las "x" y el segundo valor representa las "y".

El valor por defecto, si no se dice nada, de transform-origin: 50% 50%



TRANSFORMACIONES: SKEW

`skew(<angle>[, <angle> ])`

Aplica una operación de inclinación especificada por el vector `[ax, ay]`.

Si no se indica el valor de `ay`, su valor es `0`.

<https://developer.mozilla.org/en-US/docs/Web/CSS/transform-function/skew>

- `skewX()`: aplica una operación de inclinación en el eje x.
- `skewY()`: aplica una operación de inclinación en el eje y.



FONDOS MÚLTIPLES

En CSS3 es posible declarar varios fondos de imagen a un elemento de la página.

Hasta ahora esto era posible, creando varios elementos anidados y colocando un fondo en cada uno. Esto se puede hacer en CSS3 con un solo elemento, al que aplicaremos varios fondos distintos.

El HTML del ejemplo de varias imágenes de fondo sería el siguiente:

```
<div id="fondos">
```

```
  texto de un único elemento
```

```
  ...
```

```
</div>
```



FONDOS MÚLTIPLES

Ahora veamos el CSS válido para este ejemplo:

```
#fondos{  
  
background: url(fondo3.png) bottom right no-repeat,  
  
url(fondo2.png) center no-repeat,  
  
url(fondo1.gif) center repeat;  
  
width: 300px;  
  
}
```

Las imágenes que declaramos se van colocando de modo que la primera aparece sobre las siguientes. Así pues, en esta declaración, fondo1.gif, que está colocada como último fondo, es la que aparece detrás del todo.



FONDOS MÚLTIPLES

También se pueden establecer varios fondos mediante la propiedad `background-image`, separando las URL por comas.

```
Elemento { background-image: url("imagen1"), url("imagen2"), ...; }
```

Podemos mediante la propiedad `background-position` indicar la posición de la imagen con respecto al contenedor.

```
Elemento {  
  
    background-image: url("imagen1"), url("imagen2"), ...;  
  
    background-position: posX posY, posX posY...;  
  
}
```



FONDOS MÚLTIPLES

La propiedad `background-size` nos permite escalar una imagen dispuesta en el fondo

Elemento { `background-size: ancho alto;` }

Además de estos valores la propiedad `background-size` admite dos valores especiales: `contain` y `cover`.

- `contain`: en este caso, la imagen es redimensionada `manteniendo sus proporciones originales`, de tal manera que cada lado sea tan grande como sea posible mientras que no exceda la longitud del lado correspondiente del contenedor. La imagen `nunca excede el tamaño de su contenedor`, por lo que si no coincide con el tamaño de su contenedor, `pueden existir zonas del fondo que no queden cubiertas` por la imagen.
- `cover`: en este otro caso, la imagen es redimensionada `manteniendo sus proporciones originales`, de tal manera que el `área completa del fondo es siempre cubierta` por la imagen de fondo. En este caso, el tamaño de la imagen siempre es `igual o superior al área del fondo a cubrir`, pudiendo `existir zonas de la imagen que no sean visibles`.



COLUMNAS MÚLTIPLES

Nos permite disponer de un bloque de texto en múltiples columnas indicando la cantidad de columnas.

```
Elemento {  
    column-count: cantidad de columnas;  
}
```

CSS3 nos permite también especificar la separación entre las columnas.

```
Elemento {  
    column-gap: valor;  
}
```

Podemos utilizar cualquier unidad para indicar la separación entre columnas (px, em etc.)



COLUMNAS MÚLTIPLES

Podemos configurar una línea separadora entre las columnas cuando utilizamos múltiples columnas.

```
Elemento {  
    column-rule: grosor estilo color;  
}
```

El grosor se establecerá con unidades de medida (px, em, rem...)

El estilo con la palabra reservada (solid, dashed, dotted...)

Y el color con palabras reservadas, código hexadecimal o rgb().



OPACIDAD

La opacidad es una característica de los objetos de no dejar pasar la luz (mientras un objeto es más opaco significa que deja pasar menos luz). Un elemento HTML dispone de la propiedad `opacity` para definir cuál es su opacidad.

```
Elemento {  
    opacity: valor;  
}
```

El valor es un número comprendido entre 0 y 1. El 0 significa que es totalmente transparente (luego no se verá nada en pantalla, pero el espacio ocupado por el elemento HTML queda reservado), el 1 significa que es totalmente opaco (no deja pasar la luz).

OPACIDAD

Cuando le asignamos una opacidad a un elemento HTML luego todos los elementos contenidos en dicho elementos heredan dicha opacidad

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec,



Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec,



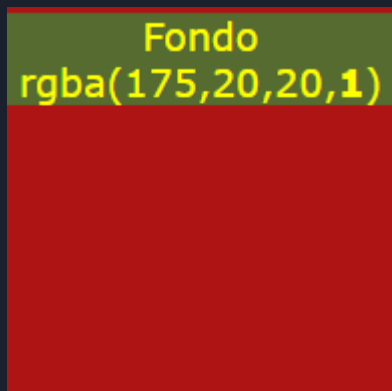
opacity: 1
opacity: 0.2

FUNCIÓN `rgba()`

La notación RGBA es una manera de especificar colores en la que se definen cuatro valores.

Los tres primeros son los bien conocidos canales RGB (rojo, verde y azul) y el cuarto parámetro es el canal **Alpha**, que no es más que el **grado de transparencia** u opacidad del color.

El canal Alpha es un valor entre cero y uno, siendo 0 totalmente transparente y 1 totalmente opaco.



Cuando usamos la función `rgba()` la opacidad se aplica sólo al elemento y no a sus hijos.